

Career Intelligence AI System – Full Mathematical & Research Mapping Brief

This document contains the full expanded explanation you requested:

- Full mapping from ICT research paper → Career Skills Graph
 - All mathematical formulas (distance, closeness, eigenvector)
 - How co-occurrence creates weighted edges
 - How shortest-path produces multi-step skill chains
 - How React → Go fallback works mathematically
 - Transition Index, CoreSkillOverlap, MarketDemand
 - Neo4j Cypher queries for demo
 - Meeting script flow
-

1. Mapping ICT Paper → Career Graph

The original ICT paper models industries as nodes and patent citations as edges. Closeness is computed using shortest paths where each edge has a weight equal to citation strength.

Our adaptation:

- SKILLS become industries.
 - JOBS become citations (a job “cites” all skills it requires).
 - Co-occurrence count = number of jobs that require two skills together.
 - Central skills behave like ICT industries: highly connected, high leverage.
-

2. Mathematical Framework

A. Edge Weight

$w(s1, s2)$ = number of jobs requiring both skills.

Example:

React & UI Design appear together in 320 job postings.

So $w(\text{React}, \text{UI Design}) = 320$.

If React & Go appear together in only 2 postings:

$w(\text{React}, \text{Go}) = 2.$

B. Edge Distance

$$d(s_1, s_2) = 1 / w(s_1, s_2)$$

Meaning:

- Higher co-occurrence = lower distance (closer skills)
- Lower co-occurrence = higher distance (farther skills)

Examples:

React–UI Design: $d = 1/320 \approx 0.003$

React–Go: $d = 1/2 = 0.5$

C. Shortest Path Distance

$D(A, B)$ = sum of distances along shortest weighted path.

Example chain:

React → UI Design → Design Systems → UX Research

Distances:

React–UI = 0.003

UI–DesignSystems = 0.004

DesignSystems–UX = 0.005

Total $D(\text{React}, \text{UX}) = 0.003 + 0.004 + 0.005 = 0.012$

D. Closeness Score

$$\text{closeness}(A, B) = 1 / (1 + D(A, B))$$

Examples:

React → UI (0.003): closeness ≈ 0.997

React → UX (0.012): closeness ≈ 0.988

React → Go (0.5): closeness ≈ 0.666

Closeness $\in [0,1]$

E. Eigenvector Centrality

Definition: Importance(s) = weighted sum of importance of neighbors.

We compute using Neo4j GDS.

Interpretation:

- A skill connected to many *important* skills becomes important.
- AI → Python, Data Structures, ML Ops → all strong neighbors → high eigenvector score.
- COBOL → few modern neighbors → low eigenvector score.

Usage:

SkillImportance(skill) = eigenvectorCentrality(skill)

This helps identify “ICT-like” core skills in the job market.

F. User→Job Skill Distance

For each required job-skill req_j:

min_distance = min(D(req_j, user_skill_i))

closeness_j = 1 / (1 + min_distance)

JobCloseness = average of closeness_j over all job skills.

G. Transition Index

TransitionIndex =

0.50 * AvgCloseness(user, target)

+ 0.30 * CoreSkillOverlap

+ 0.20 * MarketDemand

This gives a practical ranking of how feasible a transition is.

3. React → Go Example (No Direct Edge)

Step 1: Direct Edge

React–Go co-occurrence = 2

→ distance = 1/2 = 0.5 (very large distance)

Step 2: Multi-hop Search

React → JavaScript → Backend → Go

React–JS = 800

JS–Backend = 600

Backend–Go = 150

Distances:

$1/800 + 1/600 + 1/150$

$= 0.00125 + 0.00167 + 0.00667$

$= 0.00959$

This is MUCH smaller than the direct 0.5 edge if taken alone.

Conclusion:

Even if React and Go don't co-occur directly, the graph discovers a
hidden transition path via shared backend skills.

This proves the shortest-path logic works.

4. Neo4j Demo – Sample Cypher

```
CREATE (r:Skill {name:'React'})
```

```
CREATE (ui:Skill {name:'UI Design'})
```

```
CREATE (ds:Skill {name:'Design Systems'})
```

```
CREATE (ux:Skill {name:'UX Research'})
```

```
CREATE (js:Skill {name:'JavaScript'})
```

```
CREATE (be:Skill {name:'Backend'})
```

```
CREATE (go:Skill {name:'Go'})
```

```
CREATE (r)-[:CO_OCCURS_WITH {weight:320}]->(ui)
```

```
CREATE (ui)-[:CO_OCCURS_WITH {weight:250}]->(ds)
```

```
CREATE (ds)-[:CO_OCCURS_WITH {weight:200}]->(ux)
```

```
CREATE (r)-[:CO_OCCURS_WITH {weight:800}]->(js)
```

```
CREATE (js)-[:CO_OCCURS_WITH {weight:600}]->(be)
```

```
CREATE (be)-[:CO_OCCURS_WITH {weight:150}]->(go)
```

Query to visualize skills + weights:

```
MATCH (s1:Skill)-[:CO_OCCURS_WITH]->(s2)
```

```
RETURN s1, s2, r
```

Query for eigenvector centrality:

```
CALL gds.eigenvector.stream({
```

```
nodeProjection: 'Skill',
```

```
relationshipProjection: {  
  CO_OCCURS_WITH: {type:'CO_OCCURS_WITH', properties:'weight'}  
}  
})  
YIELD nodeId, score  
RETURN gds.util.asNode(nodeId).name AS skill, score  
ORDER BY score DESC
```

Query for shortest path between React and Go:

```
MATCH (a:Skill {name:'React'}), (b:Skill {name:'Go'})  
CALL gds.shortestPath.dijkstra.stream({  
  sourceNode: id(a),  
  targetNode: id(b),  
  relationshipWeightProperty: 'weight'  
})  
YIELD totalCost, nodeIds  
RETURN totalCost, nodeIds
```
